

Open Weather Flutter

Material de estudo e apresentação

Aplicativo Flutter de clima com API OpenWeather.

Inclui busca de cidades, histórico, alertas e mapa climático interativo.

Projeto organizado em camadas com Riverpod, Dio, Geolocator e Flutter Map.

Tema: Aplicativo mobile de previsão do tempo

Linguagem: Dart

Framework: Flutter

API: OpenWeather

Objetivo: Estudo, demonstração e apresentação acadêmica

Material de estudo e apresentação do projeto

Aluno: Wagner

Curso: Projeto Flutter com API OpenWeather

1. Visão geral

O Open Weather Flutter é um aplicativo mobile desenvolvido em Flutter para consultar informações meteorológicas usando a API da OpenWeather.

O objetivo do projeto é demonstrar uma aplicação real com consumo de API, gerenciamento de estado, navegação, geolocalização, busca de cidades, histórico, alertas e mapa climático interativo.

De forma prática, o usuário consegue abrir o app, permitir o uso da localização ou pesquisar uma cidade, visualizar a previsão atual, consultar alertas meteorológicos, revisar as últimas cidades buscadas e abrir um mapa com camadas de chuva, nuvens, temperatura e vento.

2. Problema resolvido

Muitos aplicativos de clima mostram apenas a temperatura atual. Este projeto busca ir além, reunindo em uma interface simples:

- clima atual;
- busca de cidade;
- alertas;
- histórico de consultas;
- mapa climático com camadas visuais.

Assim, o usuário não fica limitado a um único número de temperatura. Ele consegue entender melhor a condição meteorológica da região.

3. Principais funcionalidades

- Tela inicial com clima atual.
- Busca de cidade usando geocoding da OpenWeather.
- Atualização do clima conforme a cidade selecionada.
- Histórico das 5 últimas cidades buscadas.
- Tela de alertas meteorológicos.
- Mapa interativo com camadas OpenWeather.
- Menu lateral para navegação.

- Localização atual via GPS.
- Interface em português do Brasil.

4. Tecnologias utilizadas

O projeto utiliza Flutter e Dart como base principal.

Também foram usadas bibliotecas importantes:

- flutter_riverpod: gerenciamento de estado.
- dio: requisições HTTP.
- geolocator: acesso à localização do usuário.
- flutter_dotenv: leitura da chave da API em arquivo .env.
- flutter_map: mapa interativo leve.
- latlong2: representação de coordenadas geográficas.
- intl: formatação de datas em português.

5. Estrutura de pastas

O projeto foi organizado em camadas para separar responsabilidades.

```
lib/  
  core/  
    constants/  
    errors/  
    helpers/  
    navigation/  
  data/  
    models/  
    repositories/  
  domain/  
    providers/  
  presentation/  
    screens/  
    widgets/
```

Essa separação facilita manutenção, testes e evolução do app.

6. Papel de cada camada

core: contém elementos reutilizáveis e centrais, como constantes, navegação, helpers e tratamento de erros.

data: contém modelos e repositórios. É a camada responsável por conversar com APIs externas e transformar JSON em objetos Dart.

domain: contém providers e regras de estado da aplicação, como clima selecionado, histórico e configurações.

presentation: contém telas e widgets. É a camada visual do aplicativo.

7. Inicialização do app

O arquivo main.dart é o ponto de entrada do projeto.

Ele faz três coisas principais:

1. Inicializa os bindings do Flutter.
2. Carrega o arquivo .env com a chave da OpenWeather.
3. Inicia o app com ProviderScope, necessário para usar Riverpod.

Trecho conceitual:

```
Future<void> main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await dotenv.load(fileName: '.env');  
  runApp(const ProviderScope(child: MyApp()));  
}
```

8. Chave da API

A chave da OpenWeather não fica escrita diretamente no código.

Ela é lida a partir do arquivo .env:

```
OPENWEATHER_API_KEY=SUA_CHAVE_AQUI
```

Isso evita expor credenciais no código-fonte.

No projeto, a classe ApiConstants centraliza as URLs da API:

```
static String get apiKey =>  
  (dotenv.env['OPENWEATHER_API_KEY'] ?? '').trim();
```

9. Consumo da API OpenWeather

O app usa duas partes importantes da OpenWeather:

1. One Call API: retorna clima atual, previsão, alertas e dados por coordenada.
2. Geocoding API: transforma texto digitado pelo usuário em cidades com latitude e longitude.

O fluxo é:

```
Usuário pesquisa cidade
-> app consulta Geocoding API
-> usuário escolhe uma sugestão
-> app salva latitude e longitude
-> app consulta One Call API
-> tela atualiza com clima da cidade
```

10. Modelo de dados

O arquivo `weather_model.dart` define as classes que representam os dados meteorológicos.

Principais modelos:

- WeatherModel
- CurrentWeather
- DailyWeather
- HourlyWeather
- WeatherAlert

Esses modelos convertem o JSON da OpenWeather em objetos Dart seguros e fáceis de usar na interface.

Exemplo de responsabilidade:

```
JSON da API -> WeatherModel -> widgets da tela
```

11. Repositório de clima

O WeatherRepository é responsável pelas chamadas HTTP.

Ele usa Dio para buscar:

- clima por latitude/longitude;
- sugestões de cidades pelo nome.

Essa separação é importante porque a tela não precisa saber detalhes de URL, query parameters ou tratamento HTTP.

12. Gerenciamento de estado com Riverpod

Riverpod foi usado para controlar dados compartilhados entre telas.

Providers importantes:

- `weatherProvider`: busca e entrega o clima atual.
- `selectedLocationProvider`: guarda latitude e longitude da cidade selecionada.
- `selectedCityNameProvider`: guarda o nome da cidade selecionada.
- `historyProvider`: guarda as últimas cidades buscadas.

Com isso, quando uma cidade é selecionada, as telas que dependem do clima são atualizadas automaticamente.

13. Tela Home

A Home é a tela principal do app.

Ela mostra:

- horário atual;
- data;
- cidade;
- temperatura;
- descrição do clima;
- sensação térmica;
- umidade;
- vento;
- previsão por hora;
- previsão semanal;
- campo de busca de cidades.

Ela também é a origem do histórico: cada cidade buscada na Home pode entrar na lista das últimas cidades consultadas.

14. Busca de cidade

O campo de busca usa um `debounce`, ou seja, espera um pequeno intervalo antes de chamar a API.

Isso evita chamadas excessivas enquanto o usuário ainda está digitando.

Fluxo simplificado:

```
Usuário digita
-> app espera alguns milissegundos
-> chama API de geocoding
-> mostra sugestões
-> usuário escolhe uma cidade
-> estado global é atualizado
```

15. Histórico

A tela Histórico substituiu a antiga ideia de Favoritos.

Ela guarda as 5 últimas cidades buscadas.

Regras:

- a cidade mais recente fica no topo;
- cidades duplicadas não aparecem repetidas;
- se uma cidade já existe no histórico, ela sobe para o topo;
- ao tocar em uma cidade, a Home carrega novamente o clima daquela cidade.

O histórico atual fica em memória. Isso significa que ele reinicia quando o app é fechado.

16. Tela de alertas

A tela de alertas mostra avisos meteorológicos retornados pela OpenWeather.

Quando não existem alertas ativos, ela mostra uma mensagem amigável:

```
Nenhum alerta ativo
```

Também são exibidas dicas do dia com base nas condições atuais, como umidade elevada, vento forte ou céu aberto.

17. Mapa climático

A tela Mapa usa `flutter_map` para exibir um mapa interativo.

Ela combina:

- mapa base do OpenStreetMap;
- camada climática da OpenWeather por cima.

As camadas disponíveis são:

- Chuva;
- Nuvens;
- Temperatura;
- Vento.

Cada camada é uma URL de tiles:

```
https://tile.openweathermap.org/map/{layer}/{z}/{x}/{y}.png?appid=SUA_CHAVE
```

18. Por que flutter_map?

O flutter_map foi escolhido por ser uma solução leve.

Ele evita dependências pesadas como SDKs nativos de mapas.

Vantagens:

- funciona com tiles raster;
- é simples de configurar;
- permite sobrepor camadas;
- é suficiente para o objetivo do projeto;
- combina bem com as tiles da OpenWeather.

19. Controles do mapa

O mapa possui:

- arrastar para mover;
- gesto de pinça para aproximar;
- duplo toque para zoom;
- botões + e - para facilitar o uso;
- botão para centralizar na cidade atual;
- seleção de camada climática.

Isso melhora a usabilidade principalmente em emuladores e celulares.

20. Menu lateral

O menu lateral permite navegar entre as telas:

- Início;
- Mapa;
- Histórico;
- Alertas;
- Configurações;
- Sobre.

Ele é controlado por uma GlobalKey específica do EdgeMenu.

21. Navegação

A navegação é centralizada no AppRouter.

Rotas principais:

```
/          -> Home
/map       -> Mapa
/history   -> Histórico
/alerts    -> Alertas
/settings  -> Configurações
/about     -> Sobre
```

Centralizar rotas facilita manutenção e evita strings espalhadas pelo projeto.

22. Permissões

O app usa localização do dispositivo.

Ao iniciar sem cidade selecionada, ele solicita permissão de localização.

Se a permissão for negada, o usuário ainda pode buscar uma cidade manualmente.

23. Como rodar o projeto

Passo 1: clonar o repositório.

```
git clone https://github.com/WagnerFO/Open_Weather_API.git
```

Passo 2: entrar na pasta.

```
cd Open_Weather_API
```

Passo 3: criar o arquivo .env.

```
OPENWEATHER_API_KEY=SUA_CHAVE_AQUI
```

Passo 4: instalar dependências.

```
flutter pub get
```

Passo 5: rodar o app.

```
flutter run
```

24. Testes e qualidade

Durante o desenvolvimento foram usados:

```
flutter analyze  
flutter test
```

O flutter analyze verifica problemas de código, imports, tipos e boas práticas.

O flutter test executa os testes automatizados do projeto.

25. Desafios encontrados

Alguns desafios técnicos do projeto:

- configurar corretamente emulador Android;
- lidar com falta de espaço no emulador;
- proteger a chave da API usando .env;
- tratar permissão de localização;
- organizar estado global com Riverpod;
- evitar overflow em layouts com textos longos;
- integrar mapa com camadas de tiles.

26. Soluções aplicadas

Para o problema de espaço no emulador, foi criado um novo dispositivo virtual.

Para o overflow nos cards, foram usados Expanded, Flexible, FittedBox e TextOverflow.ellipsis.

Para o histórico, foi criado um provider específico que controla a lista das cidades recentes.

Para o mapa, foi usada uma solução leve baseada em tiles.

27. Pontos fortes do projeto

- Integração real com API externa.
- Arquitetura organizada em camadas.
- Uso de estado reativo com Riverpod.
- Interface com múltiplas telas.
- Mapa interativo com dados climáticos.
- Separação entre regra de negócio e apresentação.
- Projeto executável por outro usuário com documentação.

28. Limitações atuais

O histórico ainda não é persistido em armazenamento local.

As camadas do mapa dependem da disponibilidade dos servidores de tiles.

O app depende de uma chave válida da OpenWeather.

Alguns dados, como UV, podem ser simplificados na interface.

29. Melhorias futuras

Possíveis evoluções:

- salvar histórico localmente com SharedPreferences ou Hive;
- adicionar favoritos permanentes;
- criar modo escuro;
- melhorar tela de configurações;
- adicionar cache de clima;
- exibir detalhes de previsão diária;
- criar testes unitários para repository e providers;
- adicionar tratamento visual para falhas de internet.

30. Trechos importantes de código

Esta seção mostra partes do código que ajudam a explicar tecnicamente o projeto.

Inicialização e ProviderScope

O app carrega o arquivo .env antes de iniciar a interface. Em seguida, envolve o aplicativo com ProviderScope para habilitar o uso do Riverpod.

```
Future<void> main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await dotenv.load(fileName: '.env');  
  runApp(const ProviderScope(child: MyApp()));  
}
```

Esse trecho é importante porque garante que a chave da OpenWeather esteja disponível antes das chamadas de API.

Configuração do MaterialApp

O MaterialApp concentra configurações globais como tema, idioma, navegação e menu lateral.

```
return MaterialApp(  
  title: 'Weather App',  
  debugShowCheckedModeBanner: false,  
  theme: ThemeData(useMaterial3: true),  
  supportedLocales: const [Locale('pt', 'BR')],  
  locale: const Locale('pt', 'BR'),  
  navigatorKey: navigatorKey,  
  builder: (context, child) {  
    return EdgeMenu(key: menuKey, child: child!);  
  },  
  onGenerateRoute: AppRouter.generateRoute,  
  initialRoute: AppRouter.home,  
);
```

Aqui o builder envolve todas as telas com o EdgeMenu, permitindo abrir o menu lateral de qualquer tela.

Rotas do aplicativo

O AppRouter centraliza as rotas do projeto. Isso evita espalhar strings de navegação por várias telas.

```
class AppRouter {  
  static const String home = '/';  
  static const String map = '/map';  
  static const String history = '/history';  
  static const String alerts = '/alerts';  
  static const String settings = '/settings';  
  static const String about = '/about';  
}
```


Essa organização facilita a manutenção, porque uma alteração de rota acontece em um único lugar.

Provider de clima

O weatherProvider busca o clima com base na localização selecionada ou na localização atual do usuário.

```
final weatherProvider = FutureProvider<WeatherModel>((ref) async {
  final selectedLocation = ref.watch(selectedLocationProvider);
  final settings = ref.watch(settingsProvider);
  final repo = ref.read(weatherRepositoryProvider);

  final units = settings.tempUnit == 'C' ? 'metric' : 'imperial';
  const lang = 'pt_br';

  if (selectedLocation != null) {
    return repo.fetchWeather(
      lat: selectedLocation['lat']!,
      lon: selectedLocation['lon']!,
      units: units,
      lang: lang,
    );
  }
});
```

Na versão completa do projeto, esse fluxo também registra a cidade no histórico quando ela vem de uma busca.

Histórico das últimas cidades

O histórico usa StateNotifier para controlar uma lista de cidades recentes.

```

class HistoryNotifier extends StateNotifier<List<HistoryCity>> {
  HistoryNotifier() : super(const []);

  void add({
    required String name,
    required double lat,
    required double lon,
    required WeatherModel weather,
  }) {
    final city = HistoryCity(
      name: name,
      lat: lat,
      lon: lon,
      temp: weather.current.temp,
      description: weather.current.description,
      icon: weather.current.icon,
    );

    final withoutDuplicate = state.where((item) {
      return item.name != city.name ||
        item.lat != city.lat ||
        item.lon != city.lon;
    }).toList();

    state = [city, ...withoutDuplicate].take(5).toList();
  }
}

```

O método add remove duplicados, coloca a cidade no topo e limita a lista a 5 itens.

Busca com debounce

Na Home, a busca usa debounce para evitar muitas requisições enquanto o usuário digita.

```
Future<void> _onTextChanged(String text) async {
  final trimmed = text.trim();
  _debounce?.cancel();

  if (trimmed.length < 2) {
    setState(() {
      _suggestions = [];
      _showSuggestions = false;
      _loading = false;
    });
    return;
  }

  setState(() => _loading = true);
  _debounce = Timer(const Duration(milliseconds: 450), () async {
    await _fetchSuggestions(trimmed);
  });
}
```

Esse recurso melhora performance e evita chamadas desnecessárias à API.

Seleção de cidade

Quando o usuário escolhe uma cidade, o app atualiza providers globais.

```
void _selectCity(CitySuggestion city) {
  _controller.text = city.displayName;
  _focusNode.unfocus();
  setState(() => _showSuggestions = false);

  ref.read(selectedLocationProvider.notifier).state = {
    'lat': city.lat,
    'lon': city.lon,
  };

  ref.read(selectedCityNameProvider.notifier).state = city.displayName;
}
```

Depois disso, o weatherProvider detecta a mudança e busca o clima da nova cidade.

31. Design da interface

O design do app segue uma linha simples, limpa e funcional.

As decisões visuais principais foram:

- azul como cor principal, remetendo a clima, céu e tecnologia;
- cards arredondados para organizar informações;

- ícones climáticos para leitura rápida;
- textos curtos e objetivos;
- contraste forte entre fundo claro e cards coloridos;
- menu lateral para navegação simples.

Paleta visual

A cor principal do aplicativo é o azul `0xFF1565C0`.

Ela aparece em:

- AppBar;
- cabeçalho da Home;
- botões selecionados;
- cards de destaque;
- ícones de ação.

Esse azul foi escolhido porque comunica clima, tecnologia, céu e confiança.

Exemplo de aplicação:

```
AppBar(  
  backgroundColor: const Color(0xFF1565C0),  
  title: Text(title),  
)
```

O fundo claro `0xFFFF5F7FA` foi usado para dar contraste com os cards.

```
Scaffold(  
  backgroundColor: const Color(0xFFFF5F7FA),  
)
```

Essa combinação evita uma interface muito carregada e ajuda o usuário a focar nas informações principais.

Design da Home

A Home usa um cabeçalho azul para destacar a informação principal: cidade e temperatura.

Elementos de apoio, como umidade, vento e UV, aparecem em pequenos cards/pílulas.

Esse design ajuda o usuário a identificar rapidamente o clima atual antes de olhar os detalhes.

Estrutura visual da Home

A Home foi pensada em duas áreas:

1. Cabeçalho azul com informações principais.
2. Área inferior clara com previsão por hora ou semanal.

Essa divisão cria hierarquia visual. A parte mais importante fica no topo e os detalhes ficam abaixo.

```
Column(  
  children: [  
    _BlueHeader(weather: widget.weather),  
    Expanded(  
      child: _BottomSheet(  
        weather: widget.weather,  
        showHourly: _showHourly,  
        onTabChanged: (v) => setState(() => _showHourly = v),  
      ),  
    ),  
  ],  
)
```

O uso de `Expanded` faz a área inferior ocupar o espaço restante da tela.

Cabeçalho principal

O cabeçalho azul usa espaçamento considerando a barra de status do celular.

```
padding: EdgeInsets.only(  
  top: MediaQuery.of(context).padding.top + 12,  
  left: 20,  
  right: 20,  
  bottom: 40,  
)
```

Esse detalhe evita que o conteúdo fique atrás da câmera, notch ou barra superior do Android.

Campo de busca flutuante

O campo de busca fica posicionado entre o cabeçalho e a área inferior.

```
Positioned(  
  top: _headerHeight - 24,  
  left: 16,  
  right: 16,  
  child: const _SearchBar(),  
)
```

Esse recurso cria um efeito visual de sobreposição, deixando a tela mais moderna.

O `\_headerHeight` é calculado depois que o cabeçalho é renderizado, usando uma `GlobalKey`.

```
final box = _headerKey.currentContext?.findRenderObject() as RenderBox?;
if (box != null) {
  setState(() => _headerHeight = box.size.height);
}
```

Isso torna o posicionamento mais adaptável a telas diferentes.

Design do card de clima

O card de cidade em telas como Alertas e Histórico precisa lidar com nomes longos.

Por isso foram usadas técnicas de layout responsivo:

```
Text(
  cityName,
  maxLines: 1,
  overflow: TextOverflow.ellipsis,
)
```

Também foram usados Expanded, Flexible e FittedBox para impedir overflow em telas menores.

Evitando overflow de layout

Durante o desenvolvimento, nomes longos de cidade causaram overflow horizontal.

A solução foi combinar:

- `Expanded` para ocupar apenas o espaço disponível;
- `TextOverflow.ellipsis` para cortar texto grande;
- `Flexible` para permitir adaptação;
- `FittedBox` para reduzir elementos quando necessário.

Exemplo simplificado:

```

Row(
  children: [
    Expanded(
      child: Text(
        cityName,
        maxLines: 1,
        overflow: TextOverflow.ellipsis,
      ),
    ),
    Flexible(
      child: FittedBox(
        fit: BoxFit.scaleDown,
        child: Text('${temp.round()}°'),
      ),
    ),
  ],
)

```

Essa é uma decisão importante de design responsivo, porque o app precisa funcionar em telas pequenas e grandes.

Design do menu lateral

O menu lateral funciona como navegação global.

Cada item possui:

- ícone;
- texto;
- área clicável grande;
- cor de destaque.

Exemplo:

```

_MenuItem(
  icon: Icons.history,
  label: 'Histórico',
  onTap: () => onNavigate(AppRouter.history),
)

```

Esse padrão torna a navegação consistente entre as telas.

Design do AppBar

O AppBar personalizado foi criado no widget `AppBarMenu`.

Ele padroniza:

- cor;
- título;
- botão de menu;
- remoção do botão de voltar automático;
- estilo de texto.

```
class AppBarMenu extends StatelessWidget implements PreferredSizeWidget {
  final String title;

  const AppBarMenu({super.key, required this.title});

  @override
  Size get preferredSize => const Size.fromHeight(kToolbarHeight);
}
```

Como várias telas usam a mesma AppBar, esse widget evita repetição de código.

Design da tela Histórico

A tela Histórico usa cards verticais coloridos, parecidos com cartões meteorológicos.

Cada card mostra:

- temperatura;
- nome da cidade;
- descrição do clima;
- ícone do clima.

Esse formato é mais visual do que uma lista comum de texto.

Cards do Histórico

Cada card do histórico foi desenhado para lembrar um cartão de previsão.

O layout usa `InkWell` para toque, `Ink` para manter o efeito visual e `BoxDecoration` para cor e bordas.

```
InkWell(
  borderRadius: BorderRadius.circular(8),
  onTap: onTap,
  child: Ink(
    padding: const EdgeInsets.all(16),
    decoration: BoxDecoration(
      color: _cardColor(city.icon),
      borderRadius: BorderRadius.circular(8),
    ),
    child: Row(...),
  ),
)
```

A cor do card muda conforme o ícone do clima:

```
Color _cardColor(String icon) {
  if (icon.startsWith('09') ||
      icon.startsWith('10') ||
      icon.startsWith('11')) {
    return const Color(0xFF38679F);
  }
  if (icon.startsWith('03') ||
      icon.startsWith('04') ||
      icon.startsWith('50')) {
    return const Color(0xFF7291AE);
  }
  return const Color(0xFF42A5F5);
}
```

Isso cria uma leitura visual rápida: chuva, nuvens e céu limpo ganham tons diferentes.

Design da tela Alertas

A tela de alertas usa cards claros e separa o conteúdo em blocos.

Quando não há alerta ativo, a interface mostra um estado vazio amigável, em vez de deixar a tela sem informação.

```
if (weather.alerts.isEmpty) ...[
  const _EmptyCard(),
  const SizedBox(height: 16),
  _TipsCard(weather: weather),
]
```

Essa abordagem melhora a experiência do usuário porque a tela sempre responde com algum conteúdo útil.

Dicas do dia

As dicas são montadas dinamicamente a partir das condições atuais.

```
if (current.humidity > 80) {  
  tips.add({  
    'icon': Icons.water_drop_outlined,  
    'title': 'Umidade elevada',  
    'desc': 'Umidade em ${current.humidity}%. Hidrate-se bem.',  
  });  
}
```

Esse trecho mostra uma interface que reage aos dados, não apenas exibe texto fixo.

Design do mapa

O mapa ocupa praticamente toda a tela, porque o conteúdo principal é geográfico.

Os controles ficam sobrepostos ao mapa:

- seletor de camada no topo;
- botão de centralizar;
- botões de zoom no canto inferior direito;
- atribuição no canto inferior esquerdo.

Isso evita criar uma tela poluída e deixa mais espaço para visualizar as camadas climáticas.

Controles sobrepostos no mapa

Os controles do mapa ficam dentro de um card branco sobreposto.

```
Positioned(  
  left: 16,  
  right: 16,  
  top: 16,  
  child: _MapControls(...),  
)
```

O uso de `Positioned` dentro de `Stack` permite que o mapa continue ocupando toda a tela, enquanto os controles aparecem por cima.

```
Stack(  
  children: [  
    FlutterMap(...),  
    Positioned(child: _MapControls(...)),  
    Positioned(child: _ZoomControls(...)),  
  ],  
)
```

Essa técnica é comum em apps de mapa, pois mantém o foco no conteúdo geográfico.

Chips de camada

As camadas de clima são exibidas como chips horizontais.

```
ListView.separated(  
  scrollDirection: Axis.horizontal,  
  itemCount: _WeatherLayer.values.length,  
  itemBuilder: (context, index) {  
    final layer = _WeatherLayer.values[index];  
    return _LayerChip(  
      layer: layer,  
      selected: selectedLayer == layer,  
      onTap: () => onLayerChanged(layer),  
    );  
  },  
)
```

Esse formato é melhor do que botões grandes porque economiza espaço em telas pequenas.

Estado visual de seleção

O chip selecionado muda de cor.

```
color: selected  
  ? const Color(0xFF1565C0)  
  : const Color(0xFFE8F1FA)
```

Isso dá feedback imediato para o usuário sobre qual camada está ativa.

Botões de zoom

Os botões de zoom foram colocados no canto inferior direito.

```
Positioned(  
  right: 16,  
  bottom: 52,  
  child: _ZoomControls(  
    onZoomIn: () => _changeZoom(1),  
    onZoomOut: () => _changeZoom(-1),  
  ),  
)
```

Mesmo o mapa aceitando gesto de pinça, os botões melhoram o uso em emuladores e dispositivos com tela pequena.

32. Componentização visual

Um ponto importante do design do código foi dividir a interface em widgets menores.

Exemplos:

- `\_BlueHeader`
- `\_SearchBar`
- `\_BottomSheet`
- `\_WeatherRow`
- `\_HistoryCard`
- `\_MapControls`
- `\_LayerChip`
- `\_ZoomControls`

Essa componentização facilita:

- leitura do código;
- manutenção;
- testes;
- reaproveitamento visual;
- correção de bugs localizados.

Exemplo de widget especializado

O `\_WeatherRow` representa uma linha de previsão.

```
class _WeatherRow extends StatelessWidget {  
  final IconData icon;  
  final Color iconColor;  
  final String title;  
  final String subtitle;  
  final String rightText;  
  final bool highlight;  
}
```

Em vez de repetir o layout da linha várias vezes, o app usa o mesmo widget para previsão por hora e previsão semanal.

Alternância entre previsão por hora e semanal

A alternância é feita por estado local na Home.

```
bool _showHourly = true;
```

E a interface decide qual lista mostrar:

```
Expanded(  
  child: showHourly  
    ? _HourlyView(weather: weather)  
    : _WeeklyView(weather: weather),  
)
```

Essa decisão de design deixa a tela mais compacta, pois duas informações compartilham a mesma área.

33. Design de interação

Além do visual, o projeto também considera interação.

Principais interações:

- abrir menu lateral;
- buscar cidade;
- selecionar sugestão;
- alternar previsão por hora/semanal;
- tocar em cidade do histórico;
- mudar camada do mapa;
- aproximar/distanciar mapa;
- centralizar mapa na cidade.

Feedback de carregamento

Quando o app está buscando dados, ele mostra um `CircularProgressIndicator`.

```
loading: () => const Center(  
  child: CircularProgressIndicator(  
    color: Color(0xFF1565C0),  
  ),  
)
```

Esse feedback evita que o usuário pense que o aplicativo travou.

Estados vazios

O app também tem estados vazios para telas sem conteúdo, como alertas e histórico.

Um bom estado vazio explica o que está acontecendo e orienta o usuário.

```
const Text(  
  'Busque uma cidade na tela inicial para criar o histórico.',  
  textAlign: TextAlign.center,  
)
```

34. Boas práticas usadas no código

Algumas boas práticas aplicadas:

- separação entre tela, estado e repository;
- uso de providers para dados compartilhados;
- uso de modelos para converter JSON;
- uso de constantes para URLs de API;
- uso de widgets menores;
- proteção da API key com .env;
- tratamento de loading, erro e dados;
- uso de ellipsis para textos longos;
- uso de componentes reutilizáveis.

Tratamento de estados assíncronos

O Riverpod entrega um AsyncValue, tratado com `when`.

```
weatherAsync.when(  
  loading: () => const Center(child: CircularProgressIndicator()),  
  error: (e, _) => Center(child: Text(e.toString())),  
  data: (weather) {  
    return ListView(...);  
  },  
)
```

Isso deixa explícito o que acontece em cada estado da requisição.

Centralização de constantes

As URLs da API ficam em `ApiConstants`.

```
static String weatherTileLayer(String layer) =>  
  'https://tile.openweathermap.org/map/$layer/{z}/{x}/{y}.png'  
  '?appid=$apiKey';
```


Se a URL da OpenWeather mudar, a manutenção acontece em um único arquivo.

35. Código do mapa interativo

A tela de mapa usa FlutterMap como widget principal.

```
FlutterMap(  
  mapController: _mapController,  
  options: MapOptions(  
    initialCenter: center,  
    initialZoom: 7,  
    minZoom: 3,  
    maxZoom: 12,  
  ),  
  children: [  
    TileLayer(  
      urlTemplate: 'https://tile.openstreetmap.org/{z}/{x}/{y}.png',  
    ),  
    TileLayer(  
      urlTemplate: ApiConstants.weatherTileLayer(  
        _selectedLayer.layerId,  
      ),  
    ),  
    MarkerLayer(  
      markers: [  
        Marker(  
          point: center,  
          child: const _LocationMarker(),  
        ),  
      ],  
    ),  
  ],  
)
```

O primeiro TileLayer é o mapa base. O segundo TileLayer é a camada climática da OpenWeather. O MarkerLayer marca a cidade atual.

Camadas climáticas

As camadas são representadas por um enum.

```
enum _WeatherLayer {  
  precipitation('Chuva', 'precipitation_new', Icons.water_drop_outlined),  
  clouds('Nuvens', 'clouds_new', Icons.cloud_outlined),  
  temperature('Temp.', 'temp_new', Icons.thermostat_outlined),  
  wind('Vento', 'wind_new', Icons.air);  
}
```

Isso deixa o código mais organizado do que usar várias strings soltas na tela.

Botões de zoom

Os botões + e - controlam o zoom com MapController.

```
void _changeZoom(double delta) {  
  final camera = _mapController.camera;  
  final nextZoom = (camera.zoom + delta).clamp(3.0, 12.0);  
  _mapController.move(camera.center, nextZoom);  
}
```

O clamp impede que o zoom ultrapasse os limites definidos no mapa.

36. Fluxo completo de dados

O fluxo geral do app pode ser entendido assim:

```
Usuário digita cidade  
-> Home chama WeatherRepository.fetchCitySuggestions  
-> OpenWeather Geocoding API retorna sugestões  
-> Usuário seleciona uma sugestão  
-> selectedLocationProvider é atualizado  
-> weatherProvider busca o clima  
-> WeatherRepository chama One Call API  
-> WeatherModel é criado  
-> Telas são atualizadas  
-> Histórico salva a cidade
```

Esse fluxo mostra como interface, estado e API trabalham juntos.

37. Roteiro para apresentação oral

1. Apresentar o objetivo do app.
2. Mostrar a Home e a busca de cidades.
3. Explicar que a OpenWeather retorna dados por coordenadas.
4. Mostrar o histórico das últimas cidades buscadas.
5. Mostrar os alertas meteorológicos.
6. Mostrar o mapa e trocar entre chuva, nuvens, temperatura e vento.
7. Explicar rapidamente a arquitetura em camadas.
8. Mostrar o uso de Riverpod.
9. Explicar o arquivo .env e a proteção da chave da API.
10. Finalizar com desafios e melhorias futuras.

38. Conclusão

O projeto demonstra a construção de um aplicativo Flutter completo, com integração de API, estado global, navegação, geolocalização e mapa interativo.

Ele é um bom exemplo acadêmico porque une conceitos importantes de desenvolvimento mobile com um caso de uso real e fácil de entender: consulta climática.

Além disso, a organização em camadas permite que o projeto continue evoluindo com novas funcionalidades sem tornar o código difícil de manter.